

Reporte Tcnico: Implementacin de una Plataforma de Observabilidad con Grafana, Loki y Promtail Prctica: SEGG-U2-P4H-1

Nombre: Saul Noe Perez Aguilera

Materia: Seguridad en el desarrollo de aplicaciones

Docente: Gerardo Ramirez Villareal

Fecha: 11 de julio de 2026

Índice de contenidos

1. 1. Objetivo
2. 2. Desarrollo y Evidencias
3. 3. Actividad de Análisis: Seq vs Grafana + Loki
4. 4. Reto: Clasificación y Filtrado mediante Labels
5. 5. Pregunta de Reflexión

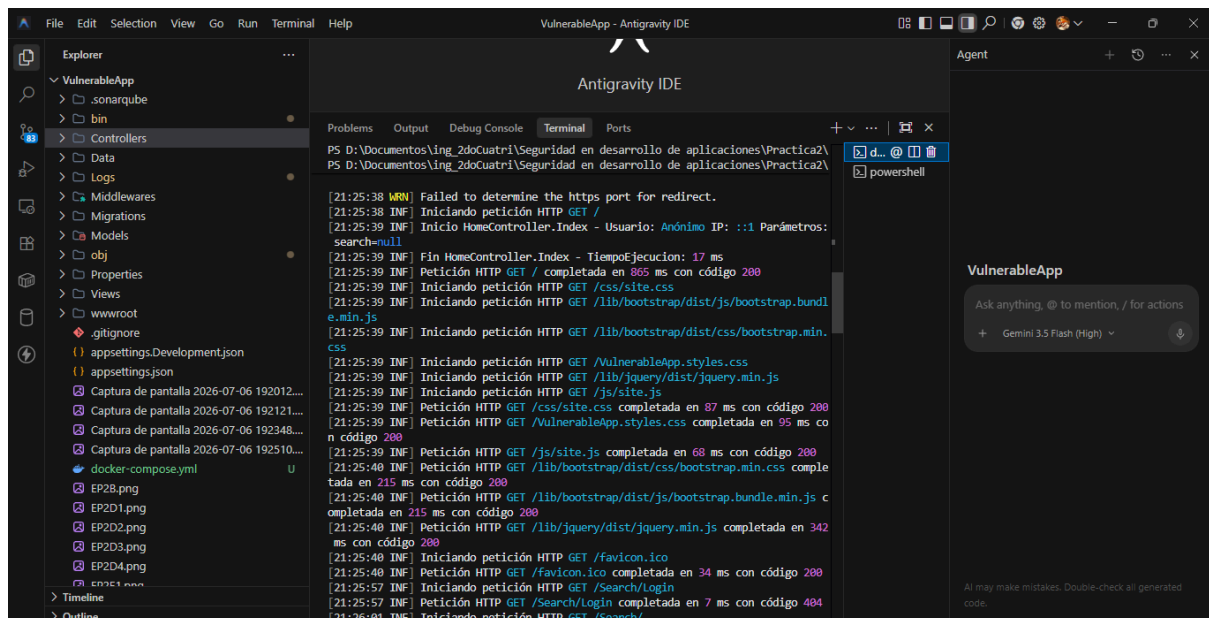
1. Objetivo

Extender la solución de logging implementada en la práctica SEGG-U2-P3G-1 incorporando Grafana, Loki y Promtail para centralizar y visualizar los registros generados por VulnerableApp, sin modificar la aplicación ni su lógica de negocio.

2. Desarrollo y Evidencias

Durante el desarrollo de esta práctica, se mantuvo VulnerableApp ejecutándose de manera local, y se integró una plataforma de observabilidad contenerizada. A continuación se presentan las evidencias del funcionamiento:

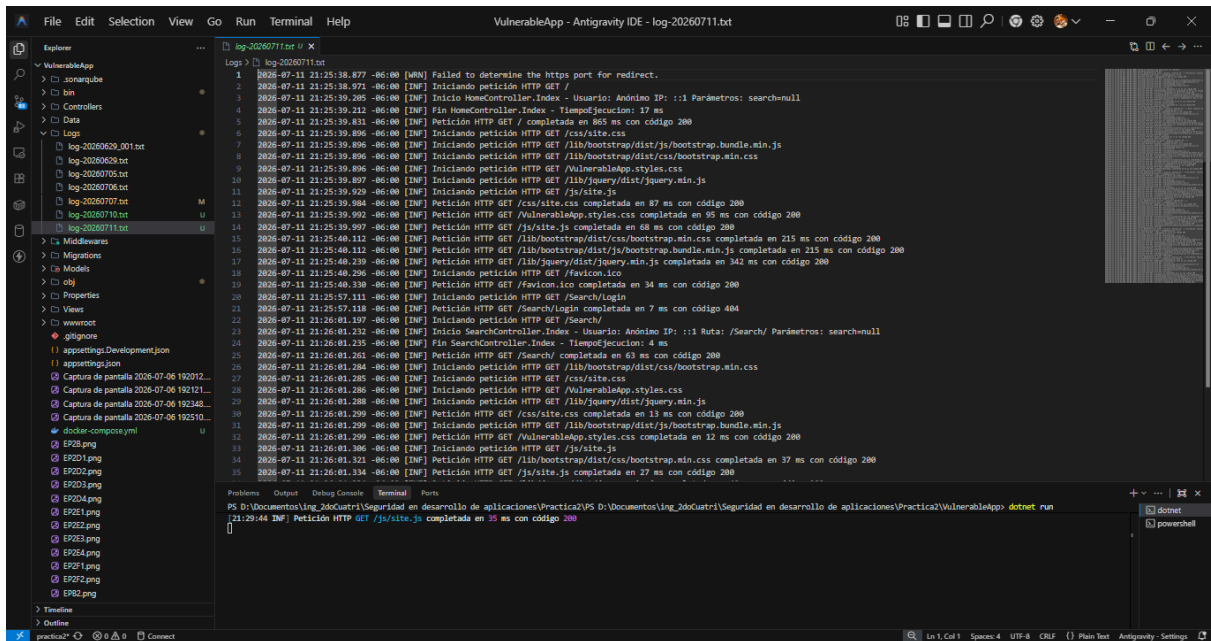
Evidencia 1: Generación de Registros A continuación se observa VulnerableApp ejecutándose y generando registros en la consola tras interactuar con sus diferentes módulos (autenticación, búsqueda, comentarios).



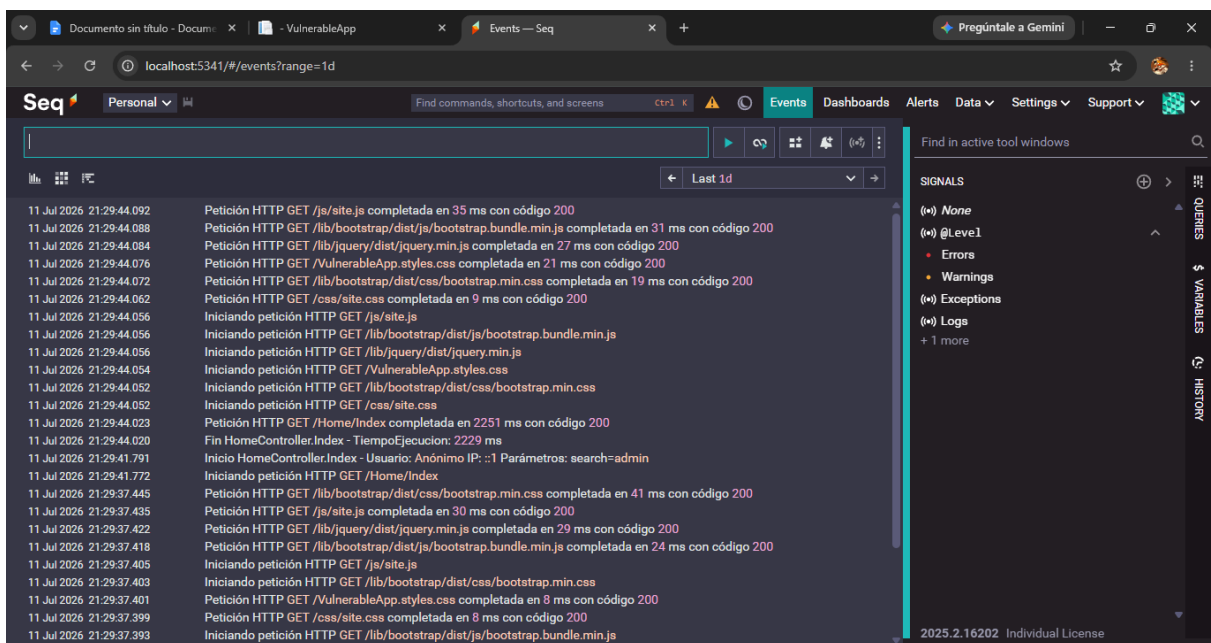
```
PS D:\Documentos\ling_2doCuatri\Seguridad en desarrollo de aplicaciones\Practica2>
PS D:\Documentos\ling_2doCuatri\Seguridad en desarrollo de aplicaciones\Practica2>

[21:25:38 WARN] Failed to determine the https port for redirect.
[21:25:38 INF] Iniciando petición HTTP GET /
[21:25:39 INF] Inicio HomeController.Index - Usuario: Anónimo IP: ::1 Parámetros: search=null
[21:25:39 INF] Fin HomeController.Index - TiempoEjecucion: 17 ms
[21:25:39 INF] Petición HTTP GET / completada en 865 ms con código 200
[21:25:39 INF] Iniciando petición HTTP GET /css/site.css
[21:25:39 INF] Iniciando petición HTTP GET /lib/bootstrap/dist/js/bootstrap.bundle.min.js
[21:25:39 INF] Iniciando petición HTTP GET /lib/bootstrap/dist/css/bootstrap.min.css
[21:25:39 INF] Iniciando petición HTTP GET /lib/jquery/dist/jquery.min.js
[21:25:39 INF] Petición HTTP GET /css/site.css completada en 87 ms con código 200
[21:25:39 INF] Petición HTTP GET /lib/bootstrap/dist/css/bootstrap.min.css completada en 95 ms con código 200
[21:25:39 INF] Petición HTTP GET /js/site.js completada en 68 ms con código 200
[21:25:40 INF] Petición HTTP GET /lib/bootstrap/dist/css/bootstrap.min.css completada en 215 ms con código 200
[21:25:40 INF] Petición HTTP GET /lib/bootstrap/dist/js/bootstrap.bundle.min.js completada en 215 ms con código 200
[21:25:40 INF] Petición HTTP GET /lib/jquery/dist/jquery.min.js completada en 342 ms con código 200
[21:25:40 INF] Iniciando petición HTTP GET /favicon.ico
[21:25:40 INF] Petición HTTP GET /favicon.ico completada en 34 ms con código 200
[21:25:57 INF] Iniciando petición HTTP GET /Search/Login
[21:25:57 INF] Petición HTTP GET /Search/Login completada en 7 ms con código 404
[21:26:01 INF] Iniciando petición HTTP GET /Search/
```

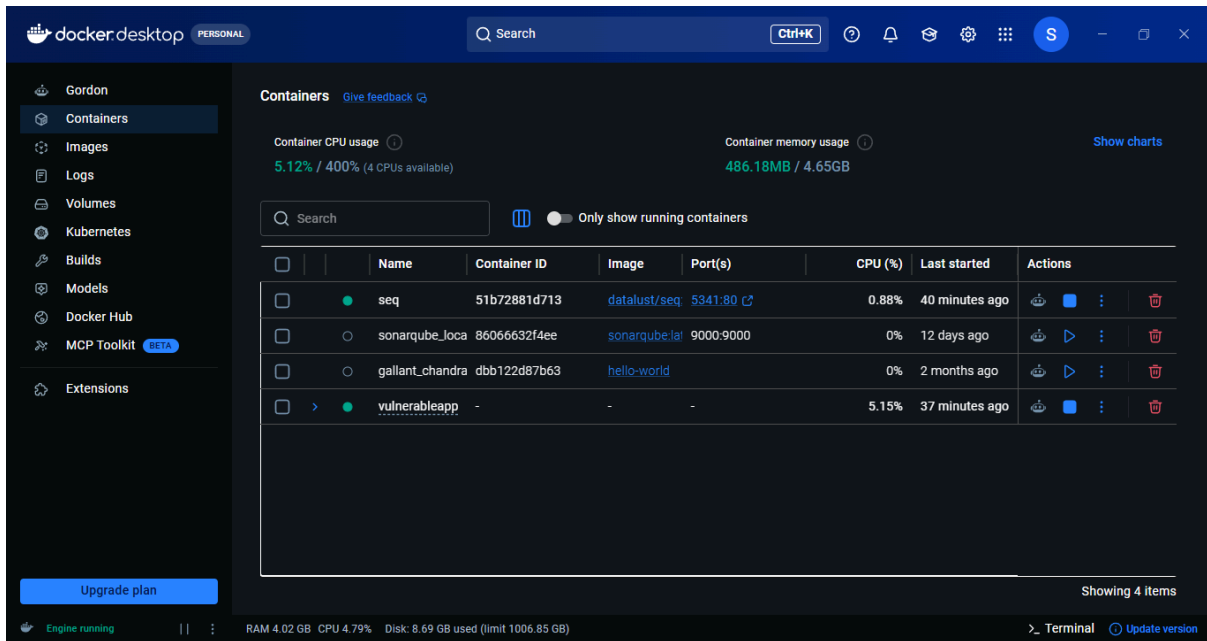
Evidencia 2: Almacenamiento en Archivos Locales Se configuró un bind mount para compartir los logs. Aquí se comprueba que los archivos de texto .txt se están generando correctamente dentro de la carpeta Logs del proyecto.



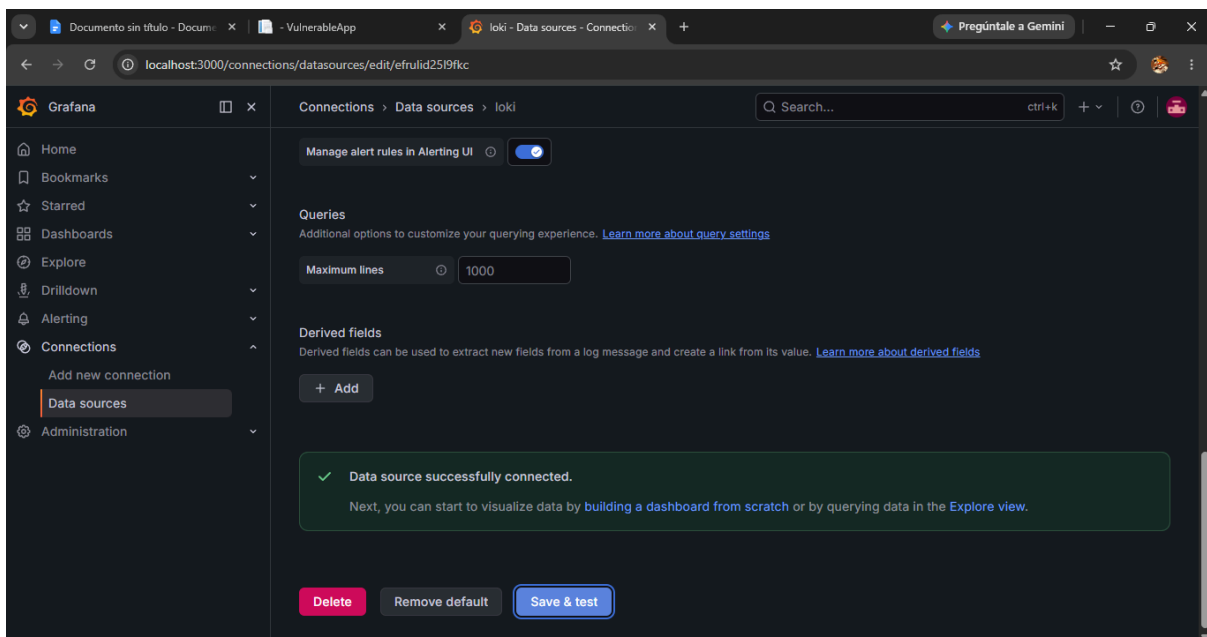
Evidencia 3: Visualización en Seq La aplicación sigue enviando registros a Seq de manera simultánea sin verse afectada por la nueva infraestructura.



Evidencia 4: Contenedores en Ejecución (Docker) La infraestructura de observabilidad se levantó exitosamente aislando Grafana, Loki y Promtail del aplicativo principal.



Evidencia 5: Integración Grafana - Loki Se configuró Loki como Data Source dentro de Grafana, logrando una conexión exitosa en el puerto interno 3100.



Evidencia 6: Archivos de Configuración (Infraestructura como Código) Para levantar los servicios, se creó el siguiente archivo docker-compose.yml:

```
None
version: '3.8'
services:
  loki:
```

```

    image: grafana/loki:latest
    ports:
      - "3100:3100"
    command: -config.file=/etc/loki/local-config.yaml
    volumes:
      - loki-data:/loki
  promtail:
    image: grafana/promtail:latest
    volumes:
      - /var/log:/var/log
      - ./Logs:/app/Logs
      - ./promtail-config.yml:/etc/promtail/config.yml
    command: -config.file=/etc/promtail/config.yml
    depends_on:
      - loki
  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=admin
    volumes:
      - grafana-data:/var/lib/grafana
    depends_on:
      - loki
volumes:
  loki-data:
  grafana-data:

```

3. Actividad de Análisis: Seq vs Grafana + Loki

Característica	Seq	Grafana + Loki
Búsqueda textual	Excelente y nativa, con sintaxis simple enfocada a propiedades estructuradas (estilo C#).	Muy potente mediante LogQL, permitiendo filtrado masivo por labels y expresiones regulares.

Característica	Seq	Grafana + Loki
Dashboards	Básicos, enfocados a mostrar métricas simples extraídas directamente de los logs.	Muy avanzados y altamente personalizables. Permiten unificar métricas de múltiples fuentes.
Alertas	Sencillas, configurables mediante apps o plugins integrados para notificaciones.	Complejas y escalables, integradas con Alertmanager (reglas basadas en umbrales y queries LogQL).
Consultas	Intuitiva, diseñada específicamente para desarrolladores y logs estructurados JSON.	LogQL es similar a PromQL; ideal para roles SRE/Ops, diseñado para procesar texto a gran escala.
Visualización	Minimalista, enfocada en expandir y explorar la trazabilidad de un registro individual.	Enfocada en series temporales, gráficos de barras, histogramas y analítica del comportamiento global.
Uso principal	Diagnóstico rápido, debugging y monitoreo específico de aplicaciones .NET.	Observabilidad integral de infraestructuras (Logs, Métricas y Trazas) de múltiples microservicios.

4. Reto: Clasificación y Filtrado mediante Labels

Para este reto, se configuró Promtail para diferenciar registros de VulnerableApp, Security y Audit.

1. Archivo de Configuración de Promtail

None

```
server:
  http_listen_port: 9080
  grpc_listen_port: 0
positions:
  filename: /tmp/positions.yaml
clients:
  - url: http://loki:3100/loki/api/v1/push
scrape_configs:
  - job_name: vulnerableapp
```

```

static_configs:
  - targets:
      - localhost
    labels:
      job: vulnerableapp_logs
      __path__: /app/Logs/*.txt
      category: VulnerableApp
pipeline_stages:
  - match:
      selector: '{job="vulnerableapp_logs"} |~
"(?i)(Autenticacin|Fallo de login|Intento de acceso|Logout)"'
      stages:
        - labels:
            category: "Security"
  - match:
      selector: '{job="vulnerableapp_logs"} |~
"(?i)(AddComment|ApiController|Search|Usuario: )" !~
"(?i)(Autenticacin|Fallo de login|Intento de acceso|Logout)"'
      stages:
        - labels:
            category: "Audit"

```

2. Consulta filtrada mediante labels en Grafana

[Insertar Captura de Grafana (pestaña Explore) mostrando la consulta {category="Audit"} y sus resultados]

3. Preguntas del Reto

¿Qué modificaciones realizó en Promtail? Se configuró el archivo promtail-config.yml agregando la sección pipeline_stages. Dentro de ella, se utilizó la etapa match evaluando expresiones regulares mediante LogQL (por ejemplo, buscando las palabras "Autenticación" o "Search") para analizar el contenido del registro. A través de la etapa labels, se asignó la etiqueta category con el valor "Security" o "Audit" dinámicamente según correspondiera, dejando "VulnerableApp" como categoría por defecto.

¿Cómo utiliza Loki las labels? Loki utiliza una arquitectura donde no indexa el texto completo de los logs, sino que únicamente indexa las labels (etiquetas de metadatos). Las labels actúan como el índice que organiza los flujos, lo que reduce drásticamente el uso de recursos, haciendo que el almacenamiento sea rápido y barato.

¿Qué ventajas ofrecen para realizar consultas con LogQL? La principal ventaja es el rendimiento. Al realizar consultas como {category="Audit"}, Loki utiliza las labels para descartar de inmediato todos los logs que no correspondan (sin leerlos), escaneando únicamente el texto de los fragmentos que coinciden con la etiqueta. Además, facilita la agregación de datos para crear gráficas agrupadas por origen o tipo de evento.

5. Pregunta de Reflexión

¿Qué ventajas ofrece incorporar una plataforma de observabilidad sin modificar la aplicación existente? Explique cómo esta arquitectura facilita la evolución de un sistema en producción.

Ofrece la ventaja del desacoplamiento total. La aplicación principal (VulnerableApp) se limita a escribir sus registros en un formato estándar en el sistema de archivos, manteniéndose agnóstica respecto a qué herramienta procesa esa información.

Esta arquitectura facilita la evolución en producción porque permite añadir, actualizar, o reemplazar por completo la plataforma de observabilidad (por ejemplo, migrar de Loki a Elasticsearch) cambiando únicamente el agente recolector (Promtail) sin requerir modificar una sola línea de código fuente, sin recompilar el proyecto y sin experimentar ningún tiempo de inactividad (zero downtime) en la aplicación. Separa las responsabilidades: la aplicación maneja el negocio, y la infraestructura paralela gestiona la observabilidad.